MODULE-2:

JDBC (Java Database Connectivity)

- **What is JDBC?** JDBC is an API (Application Programming Interface) in Java that allows Java applications to interact with databases. It provides classes and interfaces to connect, execute queries, and retrieve results from relational databases like MySQL, Oracle, etc.
- **Importance of JDBC in Java Programming**
    - Enables seamless database connectivity.
    - Provides a standard way to perform CRUD (Create, Read, Update, Delete) operations.
    - Makes applications dynamic by storing and retrieving data.
    - Supports multiple databases through different drivers.
- **JDBC Architecture Components**
    - **Driver Manager**: Manages database drivers and establishes connections.
    - **Driver**: Translates JDBC calls into database-specific calls.
    - **Connection**: Represents a session with the database.
    - **Statement**: Used to execute SQL queries.
    - **ResultSet**: Represents the result of a query, allowing traversal of rows.

🧪 **Lab Exercise**

Program: Connect to MySQL using JDBC

java

Copy

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.Statement;
import java.sql.ResultSet;


public class JdbcExample {
    public static void main(String[] args) {
        try {
            // Load JDBC driver
            Class.forName("com.mysql.cj.jdbc.Driver");
```

```
            // Establish connection
            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/testdb", "root", "password");

            // Create statement
            Statement stmt = con.createStatement();

            // Execute query
            ResultSet rs = stmt.executeQuery("SELECT * FROM students");

            // Process results
            while (rs.next()) {
                System.out.println(rs.getInt("id") + " " + rs.getString("name"));
            }

            // Close connection
            con.close();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Steps for Creating JDBC Connections • Theory: o Step-by-Step Process to Establish a JDBC Connection:

1. Importthe JDBCpackages
2. Register the JDBC driver
3. Openaconnection tothe database
4. Createa statement
5. Execute SQL queries
6. Process the result set
7. Closethe connection

Steps to Establish a JDBC Connection

1. **Import the JDBC packages**
    - Use import java.sql.*; to access JDBC classes and interfaces.
2. **Register the JDBC driver**
    - Load the driver class using Class.forName("com.mysql.cj.jdbc.Driver");.
3. **Open a connection to the database**
    - Use DriverManager.getConnection(url, user, password);.
4. **Create a statement**
    - Use Connection.createStatement() or Connection.prepareStatement().
5. **Execute SQL queries**
    - Use executeQuery() for SELECT, executeUpdate() for INSERT/UPDATE/DELETE.
6. **Process the result set**
    - Traverse results using ResultSet methods like next(), getString(), getInt()
7. **Close the connection**
    - Always close ResultSet, Statement, and Connection to free resources.

```
import java.sql.Connection;
import java.sql.DriverManager;

public class JdbcConnectionDemo {
   public static void main(String[] args) {
      try {
         // Step 2: Register JDBC driver
         Class.forName("com.mysql.cj.jdbc.Driver");

         // Step 3: Open a connection
         Connection con = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/testdb", "root", "password");

         // Confirmation message
         if (con != null) {
            System.out.println(" ✅ Connection established successfully!");
         }

         // Step 7: Close the connectio
          con.close();
                } catch (Exception e) {
                   e.printStackTrace();
                }
             }
          }
```

Theory: Types of JDBC Statements

- **Statement**
    - Executes simple SQL queries without parameters.
    - Suitable for static queries.
    - Example: stmt.executeQuery("SELECT * FROM students");
- **PreparedStatement**
    - Precompiled SQL statement.
    - Allows parameterized queries (placeholders ?).
    - More secure (prevents SQL injection)
    -
    - and efficient for repeated queries.
    - Example:
    - java
    - PreparedStatement ps = con.prepareStatement("SELECT * FROM students WHERE id=?");
    - ps.setInt(1, 101);
- **CallableStatement**
    - Used to call stored procedures in the database.
    - Supports input, output, and in-out parameters.

```
CallableStatement cs = con.prepareCall("{call getStudentName(?)}");
cs.setInt(1, 101);
```

Differences Between Statement, PreparedStatement, and CallableStatement

Feature	**Statement**	**PreparedStatement**	**CallableStatement**
Use Case	Simple queries	Parameterized queries	Stored procedures
Efficiency	Less efficient	More efficient (precompiled)	Efficient for procedures

Feature	Statement	PreparedStatement	CallableStatement
Security	Vulnerable to SQL injection	Prevents SQL injection	Secure
Flexibility	Static SQL only	Dynamic with parameters	

```
import java.sql.*;

import java.sql.*;

public class StatementCRUD {
   public static void main(String[] args) {
      try {
         Class.forName("com.mysql.cj.jdbc.Driver");
         Connection con = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/testdb", "root", "password");
         Statement stmt = con.createStatement();

         // Insert
         stmt.executeUpdate("INSERT INTO students VALUES (101, 'Priya')");

         // Update
         stmt.executeUpdate("UPDATE students SET name='Priyanka' WHERE id=101");

         // Select
         ResultSet rs = stmt.executeQuery("SELECT * FROM students");
         while (rs.next()) {
            System.out.println(rs.getInt("id") + " " + rs.getString("name"));
         }

         // Delete
         stmt.executeUpdate("DELETE FROM students WHERE id=101");

         con.close();
      } catch (Exception e) {
         e.printStackTrace();
      }
   }
}
```

Using PreparedStatement for Parameterized Queries

```
import java.sql.*;

public class PreparedStatementDemo {
   public static void main(String[] args) {
      try {
         Class.forName("com.mysql.cj.jdbc.Driver");
         Connection con = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/testdb", "root", "password");

         // Insert with parameters
         PreparedStatement ps = con.prepareStatement("INSERT INTO students VALUES (?, ?)");
         ps.setInt(1, 102);
         ps.setString(2, "Rahul");
         ps.executeUpdate();

         // Select with parameters
         ps = con.prepareStatement("SELECT * FROM students WHERE id=?");
         ps.setInt(1, 102);
ResultSet rs = ps.executeQuery();
         while (rs.next()) {
            System.out.println(rs.getInt("id") + " " + rs.getString("name"));
         }

         con.close();
      } catch (Exception e) {
         e.printStackTrace();
      }
   }
}
```

Theory: JDBC CRUD Operations

- **Insert**
  - Adds a new record into the database table.
  - Example SQL: INSERT INTO students VALUES (101, 'Priya');
- **Update**
  - Modifies existing records in the table.
  - Example SQL: UPDATE students SET name='Priyanka' WHERE id=101;
- **Select**
- Retrieves records from the database.
- Example SQL: SELECT * FROM students;

 Delete

- Removes records from the database.
- Example SQL: DELETE FROM students WHERE id=101;

JDBC CRUD Program:

```
import java.sql.*;

public class JdbcCRUD {
   public static void main(String[] args) {
      try {
```

```
        // Step 1: Load JDBC driver
        Class.forName("com.mysql.cj.jdbc.Driver");

        // Step 2: Establish connection
        Connection con = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/testdb", "root", "password");

        Statement stmt = con.createStatement();

        // INSERT
        stmt.executeUpdate("INSERT INTO students VALUES (101, 'Priya')");
        System.out.println("✅ Record inserted successfully");

        // UPDATE
        stmt.executeUpdate("UPDATE students SET name='Priyanka' WHERE id=101");
        System.out.println("✅ Record updated successfully");

        // SELECT
        ResultSet rs = stmt.executeQuery("SELECT * FROM students");
        System.out.println("✅ Records in table:");
        while (rs.next()) {
            System.out.println(rs.getInt("id") + " " + rs.getString("name"));
        }

        // DELETE
        stmt.executeUpdate("DELETE FROM students WHERE id=101");
        System.out.println("✅ Record deleted successfully");

        // Close connection
        con.close();
    } catch (Exception e) {
        e.printStackTrace();
    }
  }
}
```

Theory Recap

- **Insert** → Add a new record into a table.
- **Update** → Modify existing records based on a condition.
- **Select** → Retrieve records from the database.
- **Delete** → Remove records from the database.

💻 **Lab Exercise – Java Program**

This example uses **MySQL** with JDBC. Make sure you have:

MySQL installed and running.

A database (e.g., testdb) with a table students(id INT PRIMARY KEY, name

`VARCHAR(50), age INT).`

MySQL Connector JAR added to your project classpath.

```
import java.sql.*;
```

```
public class JdbcCrudDemo {
    // Database URL, username, password
    static final String URL = "jdbc:mysql://localhost:3306/testdb";
    static final String USER = "root";
    static final String PASS = "your_password";

    public static void main(String[] args) {
        try {
            // 1. Load Driver (optional in newer versions)
            Class.forName("com.mysql.cj.jdbc.Driver");

            // 2. Establish Connection
            Connection conn = DriverManager.getConnection(URL, USER,
PASS);

            // 3. INSERT Operation
            String insertSQL = "INSERT INTO students (id, name, age)
VALUES (?, ?, ?)";
            PreparedStatement insertStmt =
conn.prepareStatement(insertSQL);
            insertStmt.setInt(1, 1);
            insertStmt.setString(2, "Priya");
insertStmt.setInt(3, 22);
            insertStmt.executeUpdate();
            System.out.println("Record Inserted!");

            // 4. UPDATE Operation
            String updateSQL = "UPDATE students SET age=? WHERE id=?";
            PreparedStatement updateStmt =
conn.prepareStatement(updateSQL);
            updateStmt.setInt(1, 23);
            updateStmt.setInt(2, 1);
            updateStmt.executeUpdate();
            System.out.println("Record Updated!");

            // 5. SELECT Operation
            String selectSQL = "SELECT * FROM students";
            Statement selectStmt = conn.createStatement();
            ResultSet rs = selectStmt.executeQuery(selectSQL);
            System.out.println("Records in Table:");
            while (rs.next()) {
                System.out.println(rs.getInt("id") + " | " +
                                   rs.getString("name") + " | " +
                                   rs.getInt("age"));
            }

            // 6. DELETE Operation
            String deleteSQL = "DELETE FROM students WHERE id=?";
            PreparedStatement deleteStmt =
conn.prepareStatement(deleteSQL);
            deleteStmt.setInt(1, 1);
            deleteStmt.executeUpdate();
            System.out.println("Record Deleted!");

            // 7. Close Connection
            conn.close();

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
```

```
}
```

⚡ **How to Run**

```
1. Create database and table in MySQL:
sql
CREATE DATABASE testdb;
USE testdb;
CREATE TABLE students (
    id INT PRIMARY KEY,
    name VARCHAR(50),
    age INT
);
2. Save the Java file as JdbcCrudDemo.java.
3. Compile: javac JdbcCrudDemo.java
4. Run: java JdbcCrudDemo
```

👉 This program will **insert**, **update**, **select**, and finally **delete** a

What is ResultSet in JDBC?

- A ResultSet is an object returned by executing a SELECT query in JDBC.
- It represents a table of data generated by the query, with rows and columns.
- You can use it to read values from the database.

- **Navigating through ResultSet**

  - `next()` → Moves the cursor forward one row.
  - `previous()` → Moves the cursor backward one row.
  - `first()` → Moves the cursor to the first row.
  - `last()` → Moves the cursor to the last row.
  - `absolute(int row)` → Moves to a specific row number.
  - `beforeFirst()` / `afterLast()` → Moves cursor before the first row or after the last row.

- **Working with ResultSet to retrieve data**

  Use getter methods like `getInt`

  `(), getString(), getDouble() to fetch column values.`

  `Example: rs.getInt("id"), rs.getString("name").`

  `This program executes a SELECT query and demonstrates navigation through the ResultSet`

```
import java.sql.*;

public class ResultSetDemo {
   static final String URL = "jdbc:mysql://localhost:3306/testdb";
   static final String USER = "root";
   static final String PASS = "your_password";

   public static void main(String[] args) {
      try {
         // Load Driver
```

```
        Class.forName("com.mysql.cj.jdbc.Driver");

        // Establish Connection
        Connection conn = DriverManager.getConnection(URL, USER, PASS);

        // Create Statement with scrollable ResultSet
        Statement stmt = conn.createStatement(
            ResultSet.TYPE_SCROLL_INSENSITIVE,
            ResultSet.CONCUR_READ_ONLY
        );

        // Execute SELECT query
        ResultSet rs = stmt.executeQuery("SELECT * FROM students");

        // Navigate through ResultSet
        System.out.println("Using next():");
        while (rs.next()) {
            System.out.println(rs.getInt("id") + " | " +
                               rs.getString("name") + " | " +
                               rs.getInt("age"));
        }

        // Move to first record
        if (rs.first()) {
            System.out.println("\nFirst Record: " +
                rs.getInt("id") + " | " +
                rs.getString("name") + " | " +
                rs.getInt("age"));
        }

        // Move to last record
        if (rs.last()) {
            System.out.println("\nLast Record: " +
                rs.getInt("id") + " | " +
                rs.getString("name") + " | " +
                rs.getInt("age"));
        }

        // Move to previous record
        if (rs.previous()) {
            System.out.println("\nPrevious Record: " +
                rs.getInt("id") + " | " +
                rs.getString("name") + " | " +
                rs.getInt("age"));
        }

        // Close connection
        conn.close();

    } catch (Exception e) {
        e.printStackTrace();
```

```
    }
  }
}
```

How to Run

1. Ensure you have a students table in your testdb database.
2. Compile: javac ResultSetDemo.java
3. Run: java ResultSetDemo

This will show how to **retrieve and navigate records** using ResultSet.

 What is DatabaseMetaData?

- DatabaseMetaData is an interface in JDBC that provides information about the database itself.
- It describes the database's capabilities, structure, and supported features.

 Importance of Database Metadata in JDBC

- Helps developers understand the database without manually checking its configuration.
- Useful for writing database-independent code.
- Provides details like database product name, version, driver info, available tables, and supported SQL features.

Provides details like database product name, version, driver info, available tables, and supported SQL features.

 Common Methods of DatabaseMetaData

- getDatabaseProductName() → Returns the name of the database (e.g., MySQL, Oracle).
- getDatabaseProductVersion() → Returns the version of the database.
- getDriverName() → Returns the JDBC driver name.
- getTables() → Retrieves a list of tables in the database.
- supportsTransactions() → Checks if the database supports transactions.
- supportsStoredProcedures() → Checks if stored procedures are supported.

This program retrieves and displays metadata information about your database.

```
import java.sql.*;

public class DatabaseMetaDataDemo {
   static final String URL = "jdbc:mysql://localhost:3306/testdb";
   static final String USER = "root";
   static final String PASS = "your_password";

   public static void main(String[] args) {
      try {
         // Load Driver
         Class.forName("com.mysql.cj.jdbc.Driver");

         // Establish Connection
         Connection conn = DriverManager.getConnection(URL, USER, PASS);

         // Get DatabaseMetaData object
         DatabaseMetaData dbMeta = conn.getMetaData();

         // Display database information
         System.out.println("Database Name: " + dbMeta.getDatabaseProductName());
```

```
            System.out.println("Database Version: " + dbMeta.getDatabaseProductVersion());
            System.out.println("Driver Name: " + dbMeta.getDriverName());
            System.out.println("Driver Version: " + dbMeta.getDriverVersion());

            // Display list of tables
            System.out.println("\nTables in the Database:");
            ResultSet tables = dbMeta.getTables(null, null, "%", new String[] {"TABLE"});
            while (tables.next()) {
               System.out.println(tables.getString("TABLE_NAME"));
            }

            // Display supported SQL features
            System.out.println("\nSupports Transactions: " + dbMeta.supportsTransactions());
            System.out.println("Supports Stored Procedures: " +
dbMeta.supportsStoredProcedures());

            // Close connection
            conn.close();

         } catch (Exception e) {
            e.printStackTrace();
         }
      }
}
```

o/p:

```
Database Name: MySQL
Database Version: 8.0.36
Driver Name: MySQL Connector/J
Driver Version: 8.0.36
Tables in the Database:
students
employees
Supports Transactions: true
Supports Stored Procedures: true
```

What is ResultSetMetaData?

- ResultSetMetaData is an interface in JDBC that provides information about the structure of a ResultSet.
- It describes details about the columns returned by a query (like column count, names, and types).

Importance of ResultSet Metadata

- Helps developers analyze query results without hardcoding column details.
- Useful for writing dynamic applications that adapt to different database schemas.
- Enables generic reporting tools, debugging, and schema exploration.
- **Common Methods of ResultSetMetaData**
  - `getColumnCount()` → Returns the number of columns in the `ResultSet`.
  - `getColumnName(int column)` → Returns the name of the specified column.

- `getColumnType(int column)` → Returns the SQL type of the specified column (e.g., VARCHAR, INT).
- `getColumnTypeName(int column)` → Returns the database-specific type name.
- `isNullable(int column)` → Checks if a column allows `NULL` values.

This program retrieves and displays column names, types, and count of a ResultSet using ResultSetMetaData.

```
import java.sql.*;

public class ResultSetMetaDataDemo {
   static final String URL = "jdbc:mysql://localhost:3306/testdb";
   static final String USER = "root";
   static final String PASS = "your_password";

   public static void main(String[] args) {
      try {
         // Load Driver
         Class.forName("com.mysql.cj.jdbc.Driver");

         // Establish Connection
         Connection conn = DriverManager.getConnection(URL, USER, PASS);

         // Execute SELECT query
         String sql = "SELECT * FROM students";
         Statement stmt = conn.createStatement();
         ResultSet rs = stmt.executeQuery(sql);

         // Get ResultSetMetaData
         ResultSetMetaData rsMeta = rs.getMetaData();

         // Display column count
         int columnCount = rsMeta.getColumnCount();
         System.out.println("Total Columns: " + columnCount);

         // Display column details
         for (int i = 1; i <= columnCount; i++) {
            System.out.println("Column " + i + ": " +
               rsMeta.getColumnName(i) + " | " +
               rsMeta.getColumnTypeName(i) + " | " +
               "SQL Type: " + rsMeta.getColumnType(i));
         }

         // Close connection
         conn.close();

      } catch (Exception e) {
         e.printStackTrace();
      }
   }
```

}
If your students table has columns (id INT, name VARCHAR, age INT), the output will look like:
o/p:
Total Columns: 3
Column 1: id | INT | SQL Type: 4
Column 2: name | VARCHAR | SQL Type: 12
Column 3: age | INT | SQL Type: 4

SQL Query Examples • LabExercise: o WriteSQLqueriesfor: ♣ Inserting a record into a table. ♣ Updatingspecific fields of a record. ♣ Selecting records based on certain conditions. ♣ Deletingspecific records. o Implementthesequeries in Java using JDBC

- **Insert a record**

sql
INSERT INTO students (id, name, age) VALUES (101, 'Priyanka', 22);

- **Update specific fields**

sql
UPDATE students SET age = 23 WHERE id = '101'

- **Select records with condition**

sql
SELECT * FROM students WHERE age > 20;

- **Delete specific records**

sql
DELETE FROM students WHERE id = 101;

JDBC implementation in java

```
import java.sql.*;

public class PracticalSQLDemo {
   static final String URL = "jdbc:mysql://localhost:3306/testdb";
   static final String USER = "root";
   static final String PASS = "your_password";

   public static void main(String[] args) {
      try {
         // Load Driver
         Class.forName("com.mysql.cj.jdbc.Driver");

         // Establish Connection
         Connection conn = DriverManager.getConnection(URL, USER, PASS);

         // INSERT
         String insertSQL = "INSERT INTO students (id, name, age) VALUES (?, ?, ?)";
         PreparedStatement insertStmt = conn.prepareStatement(insertSQL);
         insertStmt.setInt(1, 101);
         insertStmt.setString(2, "Priyanka");
         insertStmt.setInt(3, 22);
         insertStmt.executeUpdate();
         System.out.println("Record Inserted!");

         // UPDATE
```

```
        String updateSQL = "UPDATE students SET age=? WHERE id=?";
        PreparedStatement updateStmt = conn.prepareStatement(updateSQL);
        updateStmt.setInt(1, 23);
        updateStmt.setInt(2, 101);
        updateStmt.executeUpdate();
        System.out.println("Record Updated!");

        // SELECT with condition
        String selectSQL = "SELECT * FROM students WHERE age > ?";
        PreparedStatement selectStmt = conn.prepareStatement(selectSQL);
        selectStmt.setInt(1, 20);
        ResultSet rs = selectStmt.executeQuery();
        System.out.println("Selected Records:");
        while (rs.next()) {
           System.out.println(rs.getInt("id") + " | " +
                        rs.getString("name") + " | " +
                        rs.getInt("age"));
        }

        // DELETE
        String deleteSQL = "DELETE FROM students WHERE id=?";
        PreparedStatement deleteStmt = conn.prepareStatement(deleteSQL);
        deleteStmt.setInt(1, 101);
        deleteStmt.executeUpdate();
        System.out.println("Record Deleted!");

        // Close connection
        conn.close();

      } catch (Exception e) {
        e.printStackTrace();
      }
   }
}
```

How to Use

Create a students table in your testdb database.

Compile: javac PracticalSQLDemo.java

Run: java PracticalSQLDemo

This program demonstrates **Insert, Update, Select with condition, and Delete** step by step.